CAMPUSFIX CAMPUS MAINTENANCE MANAGER

A PROJECT REPORT

Submitted by

SEERALAN M

(2116241501193)

in partial fulfilment of the End Semester Examination for the course

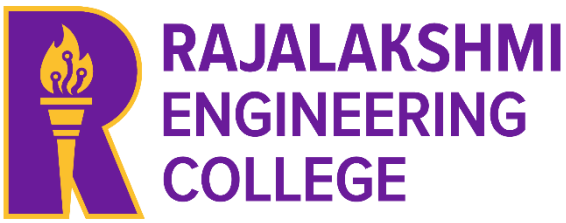
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

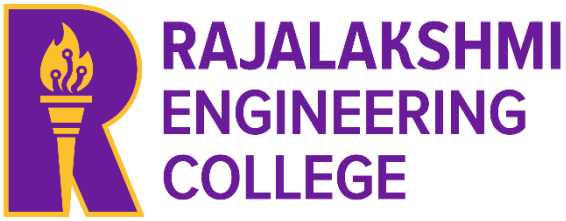


RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "CampusFix Campus Maintenance Manager" is the coursework report of Seeralan M (2116241501193) for the course CB23F35 - Low-Code Application Development using Zoho Creator, during the academic year 2026-2027, Semester V. The report describes the implemented demonstration and clearly identifies features that remain proposed or unverified.

SIGNATURE

_____

HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College

SIGNATURE

_____

COURSE FACULTY
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Seeralan M (2116241501193), submit this report titled "CampusFix Campus Maintenance Manager" for the End Semester Examination of CB23F35 - Low-Code Application Development using Zoho Creator. It records the CampusFix application in my Zoho Creator account, with implemented features distinguished from proposed extensions. The application and report were prepared with acknowledged AI assistance and the supplied academic template.

The existing Creator starter application was retained and extended with the CampusFix form, reports, dashboard and validation workflow. External documentation and assistance are acknowledged. This declaration is provided for the student to review and sign after confirming the report and institutional requirements.

Place: Chennai **SIGNATURE OF THE STUDENT**

Date: _____ **Seeralan M**

2116241501193

ACKNOWLEDGEMENT

I express my gratitude to the management of Rajalakshmi Engineering College for the infrastructure and facilities provided for academic work.

I thank the Principal of Rajalakshmi Engineering College for encouraging practical learning and application development.

I thank the Head of the Department of Artificial Intelligence and Machine Learning for the academic support extended during the course.

I thank the course faculty for the prescribed project format and learning activities that guided the scope of the application.

I acknowledge Zoho Corporation for the Creator platform and documentation. I also acknowledge the existing starter application, the supplied report template, and AI assistance used for application configuration, Deluge scripting, documentation and verification. I thank my family and friends for their support.

SEERALAN M

ABSTRACT

Campus maintenance requests often begin as messages, verbal reports or paper notes. In a demonstration scenario, this can make it difficult to locate the latest request details, know which repair team is responsible, and reconcile unresolved issues. CampusFix Campus Maintenance Manager addresses this coordination problem through a Zoho Creator application for structured request capture and progress reporting. The application was configured in the existing Creator workspace and uses a dedicated Maintenance Requests form containing twelve fields. Each request has a unique ticket reference, reporter details, location, category, priority, dates, status, assigned team, issue description and resolution notes. A form validation workflow written in Deluge checks date order, rejects future reported dates, requires a meaningful description, requires team assignment before progress beyond Reported, and requires notes before resolution. The same saved records are presented through a list report and a Kanban board grouped by four status values. An HTML and Deluge dashboard calculates total, reported, assigned, in-progress, resolved and overdue unresolved counts when loaded. Eight fictional records were entered and inspected. Live negative tests rejected the five invalid conditions and a duplicate ticket reference, while a valid record update changed the dashboard status counts as expected. The reviewed records contain one Reported request, three Assigned requests, two In Progress requests and two Resolved requests. This is an authenticated online demonstration; integrations, independent restricted-user testing, staged deployment and measured performance optimisation remain future work. The report presents the actual configuration, source logic, screenshot evidence, tests and remaining limitations, allowing the student to explain the implementation and demonstrate its behaviour without overstating production readiness.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Campus Maintenance; Request Tracking; Dashboard

PROJECT AT A GLANCE

Attribute	Details
Project Title	CampusFix Campus Maintenance Manager
Application Domain	Education / Campus maintenance
Student Name & Register No.	Seeralan M - 2116241501193
Zoho Creator Edition	Existing account; Trial has expired shown in the header
Application Name (Link Name)	CampusFix - Campus Maintenance Manager (my-first-creator-app)
Published / Live URL	https://creatorapp.zoho.in/seeralan.m.2024.aiml_raja/my-first-creator-app/#Page:CampusFix_Dashboard
Demo Credentials (per role)	Authorized owner sign-in; credentials are not published
Number of Forms / Subforms	1 project form / 0 subforms
Number of Reports (by type)	List: 1; Kanban: 1; Calendar: 0; Pivot/Chart: 0
Number of Pages / Dashboards	1 CampusFix dashboard
Number of Workflows	Form validation: 1; schedule/approval/Blueprint/batch: 0
Deluge Functions / Custom Actions	No reusable custom functions or custom report actions
Integrations	None implemented
Roles & Profiles Defined	Owner session verified; new restricted profiles not tested
Environments Used	Existing authenticated live application only
Date of Production Deployment	No staged production deployment performed

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 1
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 3
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 5
2.6 Proposed System 5
REQUIREMENT ANALYSIS 6
3.1 Stakeholders and User Roles 6
3.2 Functional Requirements 6
3.3 Non-Functional Requirements 7
3.4 Hardware and Software Requirements 7
3.5 Use Case Model 8
SYSTEM DESIGN 9
4.1 System Architecture 9
4.2 Data Model 10
4.3 Form Design 10
4.4 Process and Workflow Design 11
4.5 User Interface and Navigation Design 12
4.6 Access Control Design 12
IMPLEMENTATION 13
5.1 Application Setup 13
5.2 Forms and Data Capture 13

5.3 Reports 14
5.4 Pages and Dashboards 15
5.5 Deluge Scripting 16
5.6 Workflows and Automation 17
5.7 Integrations 17
5.8 User Management and Role-Based Access 18
5.9 AI and Analytics Features 18
5.10 Pro-Code Extension 19
5.11 Mobile and Portal Access 19
TESTING AND DEBUGGING 20
6.1 Testing Strategy 20
6.2 Test Cases 20
6.3 Debugging Deluge Scripts 21
6.4 Defect Log 22
6.5 User Acceptance Testing 22
DEPLOYMENT, SECURITY AND GOVERNANCE 23
7.1 Environments and Deployment Strategy 23
7.2 Performance Optimisation 24
7.3 Security Features 24
7.4 Governance and Administration 24
RESULTS AND DISCUSSION 26
8.1 Application Output 26
8.2 Achievement of Objectives 27
8.3 Comparison with the Existing System 28
8.4 Limitations of the Low-Code Approach 28
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 28
CONCLUSION AND FUTURE ENHANCEMENTS 29
9.1 Conclusion 29
9.2 Future Enhancements 29
9.3 Learning Outcomes and Career Relevance 29
REFERENCES 30
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 31
APPENDIX B – COURSE OUTCOME MAPPING 33
APPENDIX C – COMPLETE DELUGE SOURCE CODE 34
APPENDIX D – USER MANUAL 35
D.1 Accessing the Application 35
D.2 Instructions for the Owner Demonstration 35

D.3 Instructions for Reviewing Reports 35
APPENDIX E – EVALUATION RUBRIC.....36
APPENDIX F – VIVA-VOCE RECORD SHEET.....38

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....3
Table 2.2: Summary of Literature Survey....4
Table 3.1: Stakeholders and User Roles....6
Table 3.2: Functional Requirements....6
Table 3.3: Non-Functional Requirements....7
Table 3.4: Hardware and Software Requirements....7
Table 3.5: Use Case Description - Update a Maintenance Request....8
Table 4.1: Entities and Relationships....10
Table 4.2: Field Specification - Maintenance Requests Form....10
Table 4.3: Role–Permission Matrix....12
Table 5.1: Reports Implemented....14
Table 5.2: Deluge Scripts Implemented....16
Table 5.3: Workflows Implemented....17
Table 5.4: Integrations Implemented....17
Table 6.1: Test Cases....20
Table 6.2: Defect Log....22
Table 6.3: User Acceptance Feedback....22
Table 7.1: Deployment Checklist....23
Table 7.2: Performance Optimisation Results....24
Table 7.3: Security Controls Implemented....24
Table 8.1: Mapping of Objectives to Outcomes....27
Table 8.2: Existing System versus Proposed Application....28
Table A.1: Feature Coverage Checklist....31
Table B.1: Mapping of Course Outcomes to Report Chapters....33
Table E.1: Evaluation Rubric for the Individual Project....36
Table E.2: Performance Level Descriptors....37
Table F.1: Viva-Voce Questions and Responses....38
Table F.2: Marks Summary....38

LIST OF FIGURES

Figure 2.1: Illustrative manual maintenance request process 3
Figure 3.1: CampusFix use cases with implemented owner access and proposed roles 8
Figure 4.1: CampusFix application architecture 9
Figure 4.2: CampusFix single-form data model 10
Figure 4.3: Maintenance request validation and update flow 11
Figure 4.4: CampusFix project navigation structure 12
Figure 5.1: CampusFix dashboard and project navigation 13
Figure 5.2: Maintenance Requests in the form builder 14
Figure 5.3: Maintenance Requests in live mode 14
Figure 5.4: Repair Status Board grouped by Status 15
Figure 5.5: CampusFix Dashboard with saved-record indicators 16
Figure 5.6: Created or Edited validation workflow and saved Deluge action 17
Figure 5.7: Current Users settings and email-confirmation prerequisite 18
Figure 5.8: CampusFix count indicators used for reconciliation 19
Figure 6.1: Actual rejection of a future reported date 22
Figure 7.1: Current CampusFix application settings 25
Figure 8.1: Maintenance Requests form for issue capture 26
Figure 8.2: Saved CF-008 maintenance request details 27
Figure 8.3: Dashboard after the saved status update 27

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity-Relationship
HTML	HyperText Markup Language
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
UI	User Interface
UTC	Coordinated Universal Time

CHAPTER 1
INTRODUCTION

1.1 Background

CampusFix is an educational demonstration for a college maintenance setting. Typical issues include electrical failures, plumbing leaks, network faults, damaged furniture and cleaning requests. The intended participants are reporters, a maintenance coordinator and repair staff. In the implemented application, these responsibilities are demonstrated through the owner account rather than through separately configured user profiles. The dataset consists of fictional campus locations and example.com contact addresses.

1.2 Problem Statement

The project addresses the lack of a consistent maintenance request record and a readily reviewable status summary in an illustrative message-based process. A coordinator needs a single place to capture a request, identify its team and target date, review progress and retain the resolution explanation.

- Unstructured messages can omit the location, category or description needed to start a repair.
- Repeated follow-ups make it difficult to identify the latest assignment and status.
- Separate manual summaries can disagree with individual request records.

1.3 Objectives

- To capture maintenance requests in a twelve-field form with required and unique reference checks.
- To reject the five invalid data conditions through a Deluge validation workflow.
- To display the same saved requests in a list and a four-status Kanban board.
- To calculate six dashboard indicators from saved records and verify them against the reports.
- To test valid submission, record updates and invalid entries using fictional data, and document unimplemented extensions honestly.

1.4 Scope of the Project

The implemented scope is an authenticated browser application containing one project form, two reports, one dashboard and one form-validation workflow. It handles capture,

manual assignment, status editing and resolution notes. The original order-template components remain in the workspace but are excluded from the project counts. Separate user roles, notifications, external integrations, approval processes, mobile/portal access and staged production deployment are outside the demonstrated scope.

1.5 Justification for a Low-Code Approach

Zoho Creator provides linked form, record and report components within a hosted application [1]. For this project, visual field configuration reduces the amount of interface and storage code that must be written, while Deluge expresses the specific rejection rules [2], [3]. This supports a small instructional application whose schema and reports can be inspected in one workspace. No development-time or cost saving was measured against a separately built traditional application.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The illustrative existing process begins when a reporter notices a fault and sends a message or paper note to a coordinator. The coordinator contacts a repair team and later requests a progress update. A summary is prepared separately from those communications. Figure 2.1 is a process design illustration, not a measured study of a particular department.

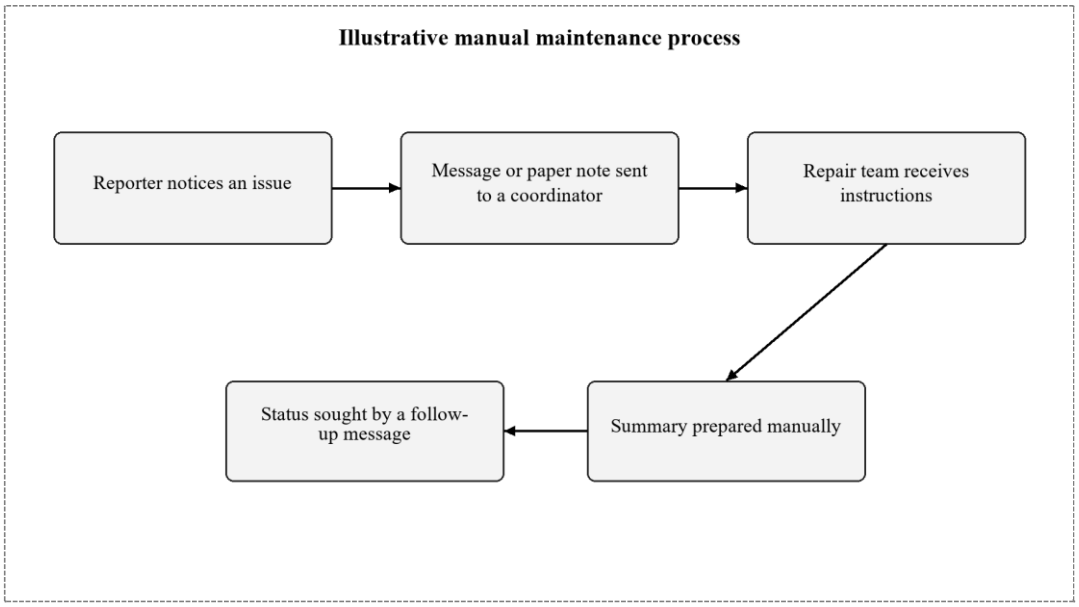


Figure 2.1: Illustrative manual maintenance request process

2.2 Limitations of the Existing System

- The initial message may not contain all information required for a repair.
- Assignments and status changes may be distributed across separate conversations.
- A manual summary must be reconciled again whenever a request changes.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Separate interface, storage and report implementation needed	Configured visual components; no time benchmark

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Skills required	Database, interface and service programming	Form/report configuration and Deluge
Hosting & infrastructure	Hosting must be selected and maintained	Existing Creator-hosted workspace
Mobile application	Separate responsive/mobile validation required	Mobile behaviour remains unverified
Security & access control	Application security must be implemented and tested	Platform permissions available; restricted roles not tested
Maintenance & change	Changes span application components	Schema and report changes managed in Creator
Customisation limits	Greater control with more code to maintain	Current dashboard uses HTML and Deluge
Cost	Not measured	Subscription limits observed; cost comparison not measured

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Zoho Corporation [1], accessed 2026	Understand Applications	Forms store data; reports and pages present it	Basis of the form/list/page architecture
2	Zoho Corporation [2], accessed 2026	Understanding form builder	Fields and properties are configured in the builder	Basis of field choices and validation properties
3	Zoho Corporation [3], accessed 2026	Understanding workflows	Events trigger defined actions	Basis of the Created or Edited validation workflow
4	Zoho Corporation [4], accessed 2026	Understanding permissions	Permission sets control component and record actions	Basis of proposed role tests, not evidence of implementation
5	Zoho Corporation [5], accessed 2026	Deluge Count	Criteria select the records counted	Basis of the six dashboard counters

2.5 Emerging Trends in Low-Code Development

CampusFix uses visual configuration for structured capture and reports, and low-code scripting for business rules. AI assistance supported configuration and documentation, but the saved definitions and live tests were checked separately. Integration, role-based collaboration and governed releases are relevant extensions; their availability in a platform does not mean they were implemented in this project.

2.6 Proposed System

The proposed system was implemented as CampusFix. A single saved request supplies the form detail, list row and status-board card. The dashboard counts those records through criteria-based aggregate queries [5]. This provides a consistent demonstration of capture, repair coordination and workload reporting without maintaining a second summary dataset.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Owner / administrator	Implemented access used in this demonstration	Configure components, submit/edit records and inspect views
Campus reporter	Proposed restricted role	Submit issues and view permitted own requests
Maintenance coordinator	Proposed restricted role	Assign teams and review allowed workload
Repair staff	Proposed restricted role	Update permitted assignments and resolution notes

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	Capture a request with required ticket, reporter, email, location, choices, dates and description	Owner demo	High
FR-02	Reject duplicate ticket references	Owner demo	High
FR-03	Reject target dates before reported dates and future reported dates	Owner demo	High
FR-04	Reject descriptions shorter than ten trimmed characters	Owner demo	High
FR-05	Require a team beyond Reported and notes for Resolved	Owner demo	High
FR-06	List saved requests and group board cards by Status	Owner demo	High
FR-07	Calculate six workload counts and reflect saved updates on reload	Owner demo	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	A small demonstration must load and reconcile; no response-time benchmark is claimed
NFR-02	Security	Authorized owner sign-in only verified; role isolation requires future tests
NFR-03	Usability	Named fields, readable statuses and navigation links in the browser
NFR-04	Availability	Requires internet and a working Creator account
NFR-05	Maintainability	Stable field link names and documented validation rules
NFR-06	Auditability	Documented source and test evidence; platform audit trail not verified

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator India data centre; existing trial-expired workspace
Scripting language	Deluge; HTML for the dashboard snippet
Browser	Internet-connected supported desktop browser
API testing tool	Not used in this implementation
Other Zoho services	No additional service connection implemented
Mobile	Not tested
Client hardware	A desktop/laptop capable of running the browser; no minimum benchmark performed

3.5 Use Case Model

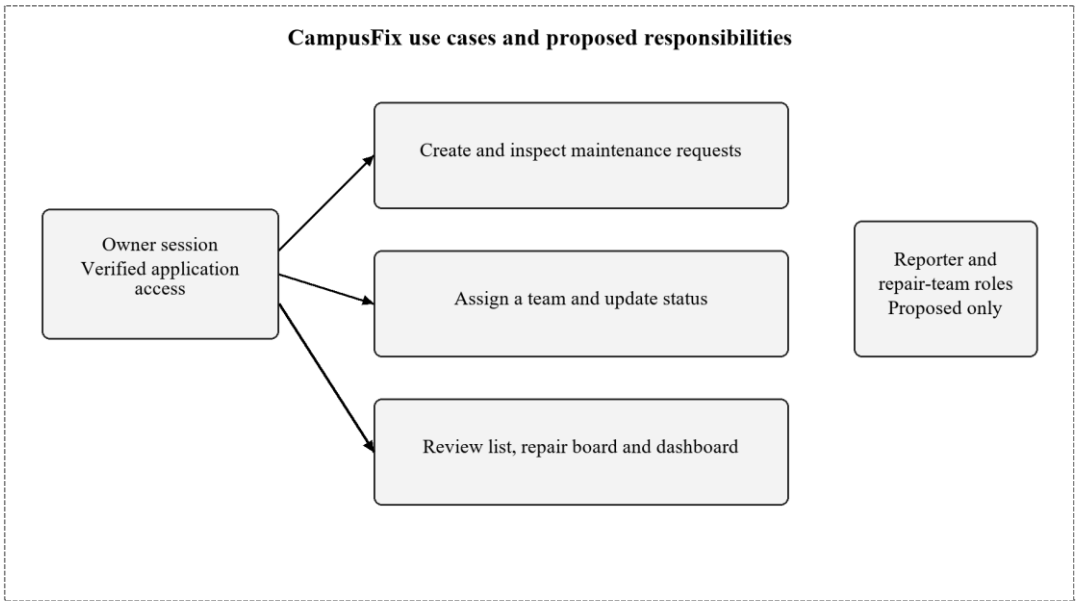


Figure 3.1: CampusFix use cases with implemented owner access and proposed roles

Table 3.5: Use Case Description - Update a Maintenance Request

Use Case ID	UC-01
Use Case Name	Update a maintenance request
Actor(s)	Owner demonstrating coordinator responsibilities
Pre-condition	Existing saved request and authorized account access
Main Flow	1. Open record. 2. Choose Edit. 3. Set team/status/notes. 4. Update.
Alternate Flow	Missing team or notes is rejected when required by status
Post-condition	Saved record is reflected in reports after reload
Related FR	FR-05, FR-06, FR-07

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

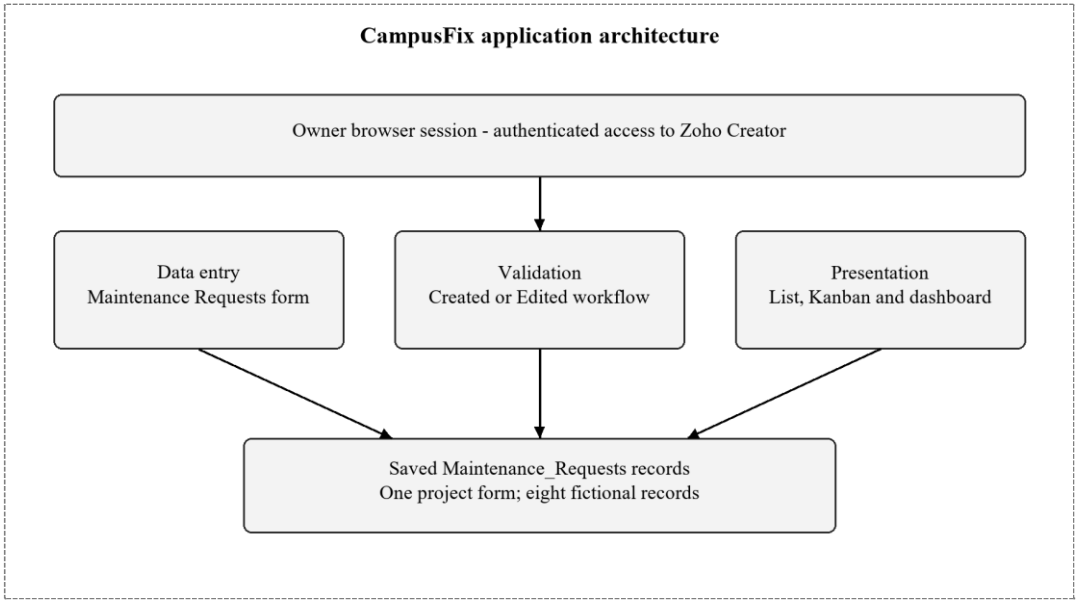


Figure 4.1: CampusFix application architecture

The browser is the access layer. The Maintenance Requests form collects data, the validation workflow checks submissions, and Creator stores the accepted records. The list, board and dashboard form the presentation layer. No external integration layer was configured. The original order components share the workspace but are not part of this maintenance model.

4.2 Data Model

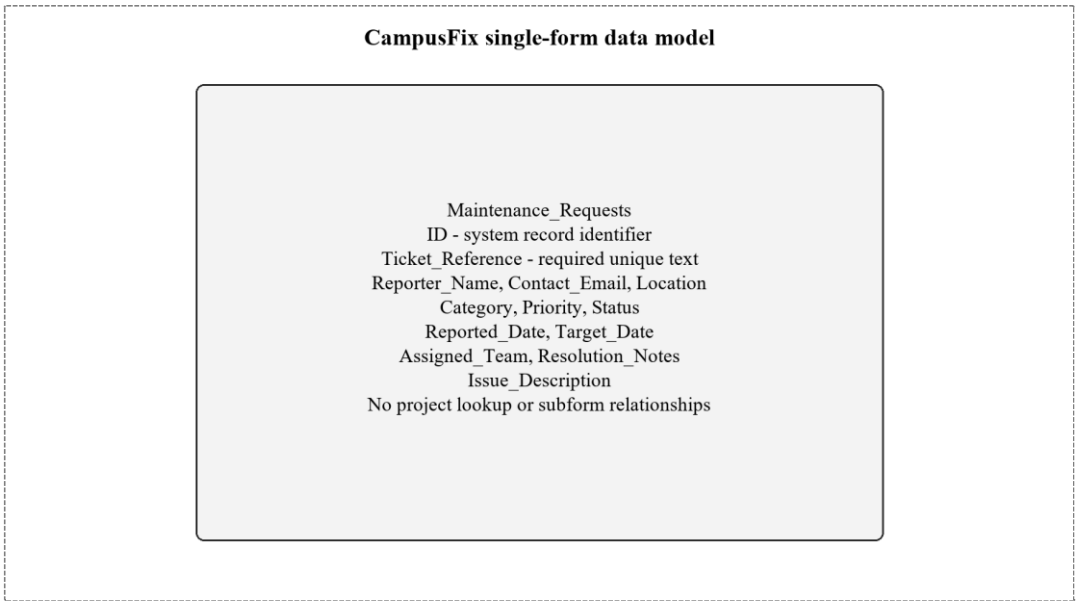


Figure 4.2: CampusFix single-form data model

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Maintenance Requests	ID; unique Ticket_Reference; reporter/location/status/dates	None in project scope	Single form	Creator saved records
Category/Priority/Status choices	Configured dropdown values	Maintenance Requests	Enumerated field choices	Drop Down fields
Original order template	Excluded from project model	Not related to CampusFix records	Retained starter content	Not counted as maintenance work

4.3 Form Design

Table 4.2: Field Specification - Maintenance Requests Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Ticket Reference	Ticket_Reference	Single Line	Yes	Unique; no duplicates	User-entered reference
Reporter Name	Reporter_Name	Single Line	Yes	Required	Fictional demo reporter
Contact Email	Contact_Email	Email	Yes	Required; personal-data flag	example.com addresses

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Location	Location	Single Line	Yes	Required	Campus issue location
Category	Category	Drop Down	Yes	Five categories	Electrical/Plumbing/Network/Furniture/Cleaning
Priority	Priority	Drop Down	Yes	Low/Medium/High	Required choice
Status	Status	Drop Down	Yes	Default Reported	Four configured values
Target Date	Target_Date	Date	Yes	Not before reported date	dd-MMM-yyyy
Reported Date	Reported_Date	Date	Yes	Not in future	dd-MMM-yyyy
Assigned Team	Assigned_Team	Single Line	Conditional	Required beyond Reported	Text; not a security role
Resolution Notes	Resolution_Notes	Multi Line	Conditional	Required for Resolved	Retains repair explanation
Issue Description	Issue_Description	Multi Line	Yes	Minimum 10 trimmed characters	Checked in workflow

4.4 Process and Workflow Design

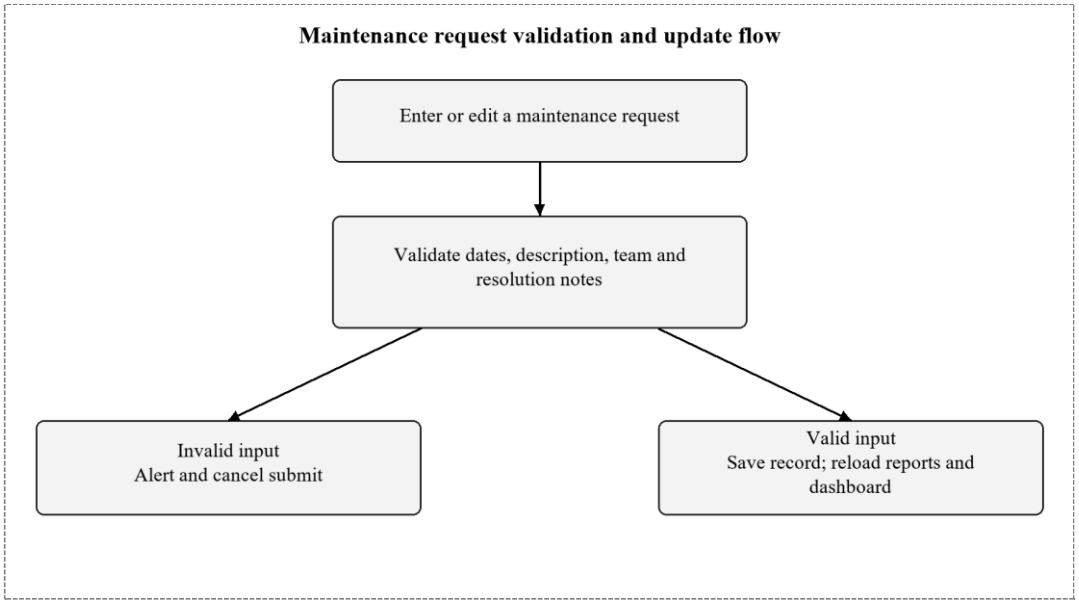


Figure 4.3: Maintenance request validation and update flow

4.5 User Interface and Navigation Design

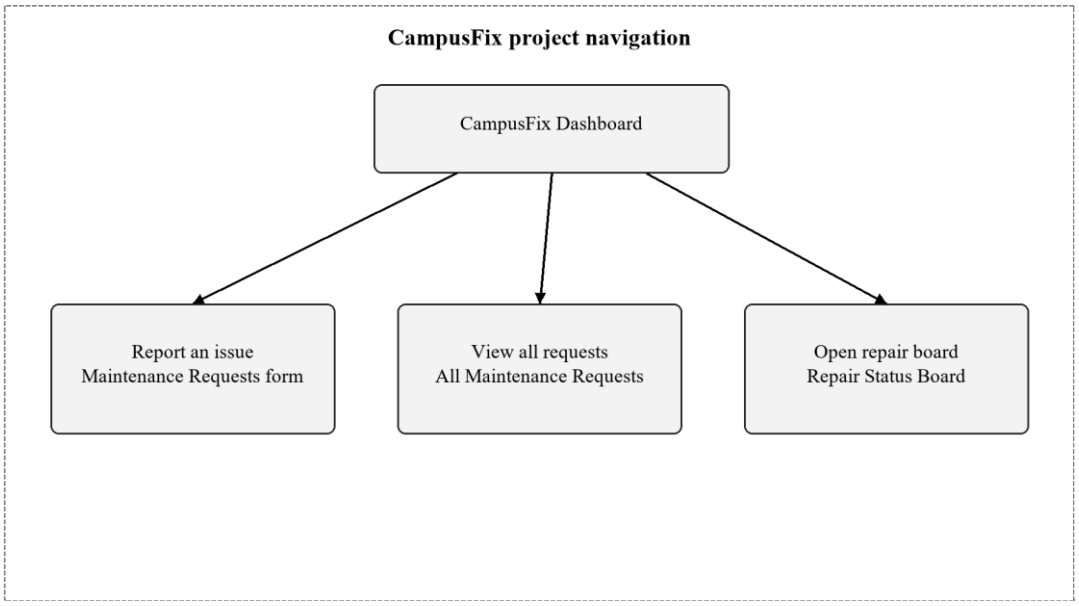


Figure 4.4: CampusFix project navigation structure

4.6 Access Control Design

Table 4.3: Role–Permission Matrix

Component	Owner session	Coordinator	Repair staff	Portal user
Maintenance Requests form	A E V D	Proposed only	Proposed only	Not configured
All Maintenance Requests	V	Proposed only	Proposed only	Not configured
Repair Status Board	V / edit record	Proposed only	Proposed only	Not configured
CampusFix Dashboard	V	Proposed only	Proposed only	Not configured

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The account already contained a starter order application. The current subscription permitted only one application, so CampusFix components were added to that workspace while the original order components were retained. The application display name is CampusFix - Campus Maintenance Manager; the existing link name remains my-first-creator-app. The project uses a teal and white HTML dashboard, with links to its form and reports.

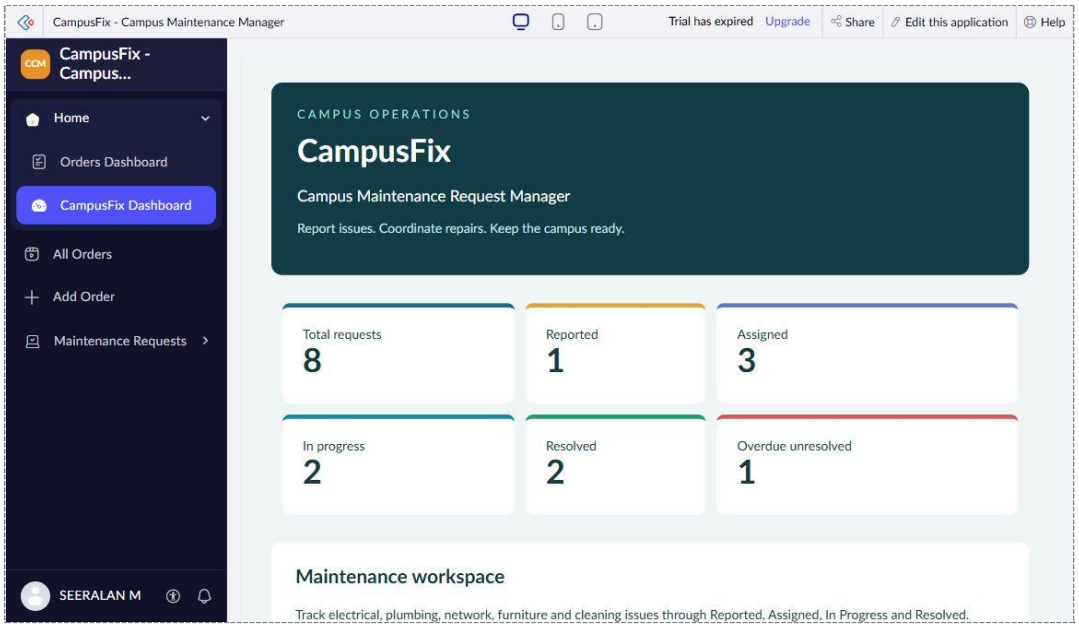


Figure 5.1: CampusFix dashboard and project navigation

5.2 Forms and Data Capture

Maintenance Requests was created from scratch in the form builder. Its twelve fields use five distinct field types: Single Line, Email, Drop Down, Date and Multi Line. Ticket Reference is mandatory and unique. Status defaults to Reported. Assigned Team and Resolution Notes are optional field controls with conditional requirements enforced by the validation workflow. Contact Email is marked as personal data in the saved definition. No project lookup or subform is present.

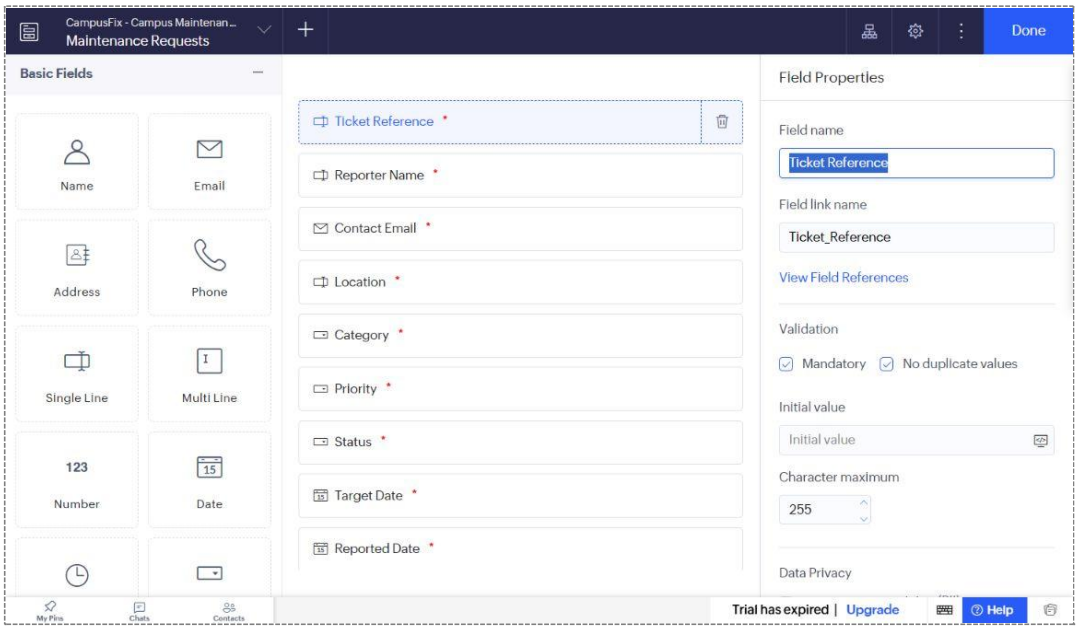


Figure 5.2: Maintenance Requests in the form builder

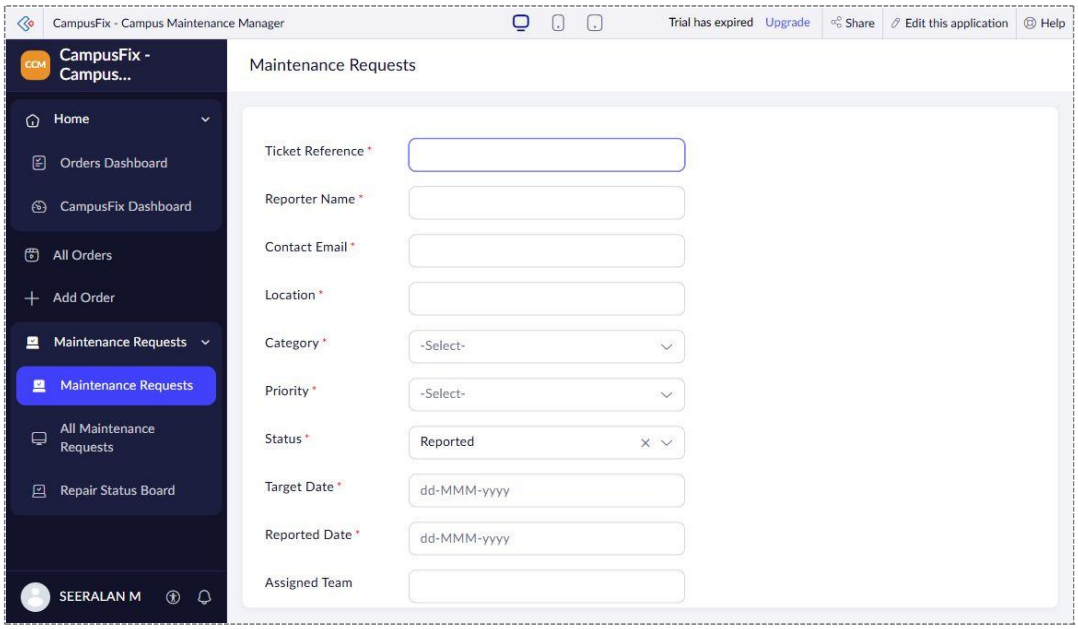


Figure 5.3: Maintenance Requests in live mode

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Maintenance Requests	List	Maintenance_Requests	Saved record listing; 8 of 8 reviewed	Standard add/edit/detail actions	Owner session

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Repair Status Board	Kanban	Maintenance_Requests	Columns grouped by Status	Open/edit record	Owner session

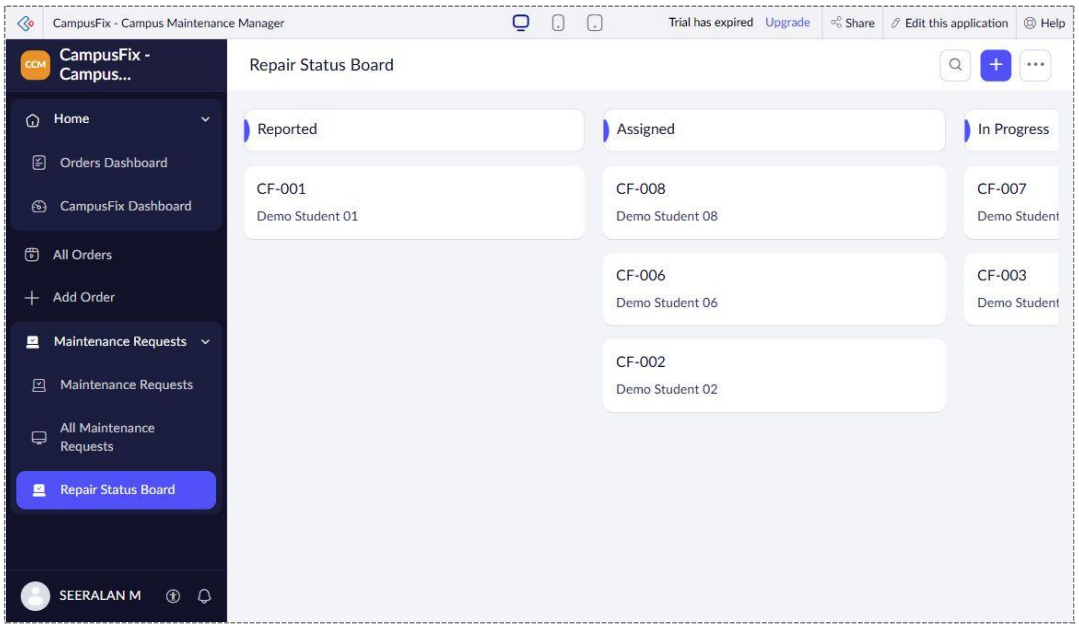


Figure 5.4: Repair Status Board grouped by Status

5.4 Pages and Dashboards

CampusFix Dashboard is a page containing an HTML and Deluge snippet. It presents six indicators and links to Report an issue, Open repair board and View all requests. The values are evaluated from saved Maintenance_Requests records when the page loads. The November 2 review showed 8 total, 1 Reported, 3 Assigned, 2 In Progress, 2 Resolved and 1 overdue unresolved. The overdue indicator is a subset of unresolved requests and changes with the current date.

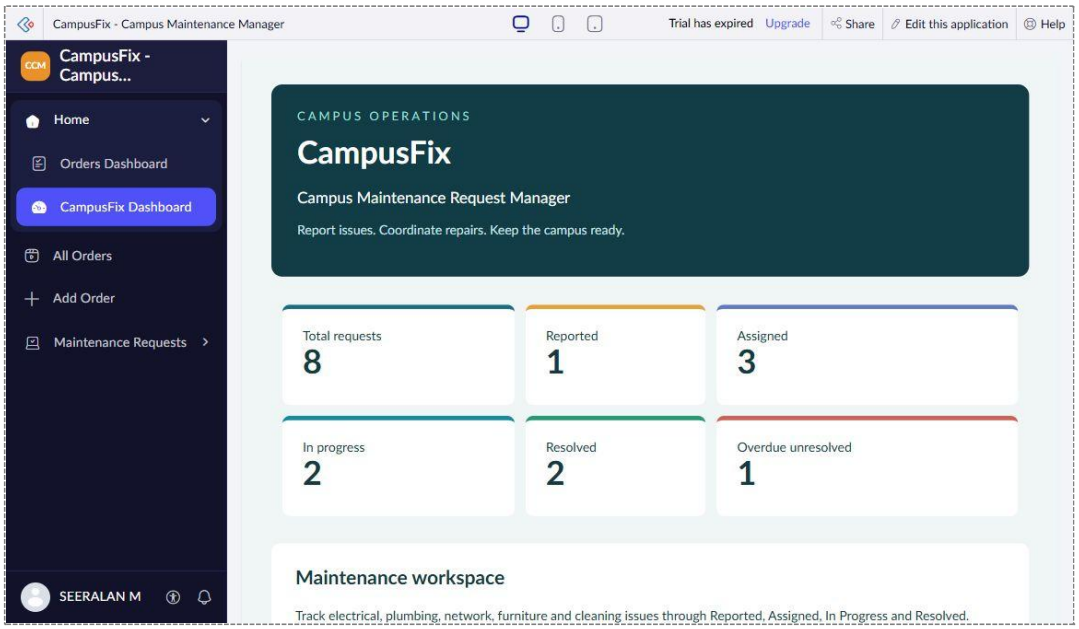


Figure 5.5: CampusFix Dashboard with saved-record indicators

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Maintenance Requests - Created or Edited, On Validate	Reject invalid submissions	if, ifnull, trim, date comparison, alert, cancel submit
DS-02	CampusFix Dashboard HTML snippet - page load	Calculate six counts	criteria fetch, count, current date

Code 5.1: DS-01 Maintenance request validation excerpt

```
// DS-01 excerpt: reject invalid target date
if(input.Target_Date < input.Reported_Date)
{
    alert "Target date must be on or after the reported date.";
    cancel submit;
}
```

Code 5.2: DS-02 Dashboard aggregate queries

```
// DS-02: dashboard aggregation
total = Maintenance_Requests[ID != 0].count();
reported = Maintenance_Requests[Status == "Reported"].count();
assigned = Maintenance_Requests[Status == "Assigned"].count();
progress = Maintenance_Requests[Status == "In Progress"].count();
resolved = Maintenance_Requests[Status == "Resolved"].count();
overdue = Maintenance_Requests[
    Target_Date < zoho.currentdate && Status != "Resolved"
].count();
```

DS-01 runs before record submission on creation or editing. It uses if statements, null-safe string checks, date comparisons, alerts and cancel submit. DS-02 uses criteria-based count queries; it does not change records. The complete validation source and all six queries are provided in Appendix C. No notification script was implemented.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Validate Maintenance Requests	Form validation	Created or Edited; form submission	Five invalid input conditions	Show alert; cancel submission	DS-01

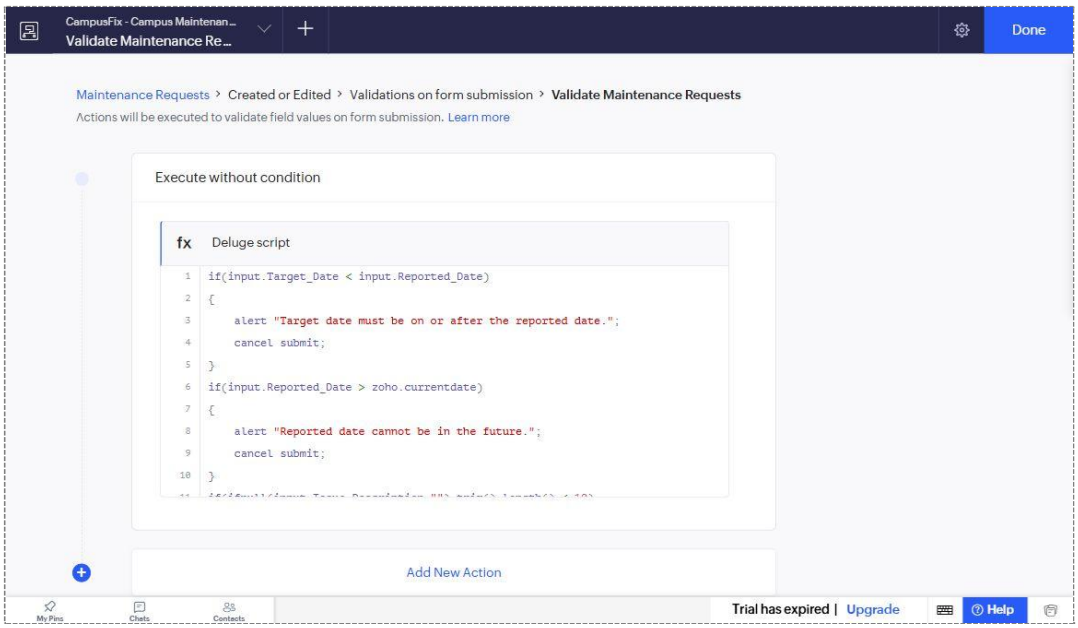


Figure 5.6: Created or Edited validation workflow and saved Deluge action

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
None implemented	No external endpoint configured	Not applicable	No connection or token created	Application uses its own saved records

No Zoho-service connection, external REST request, webhook or Creator API integration was implemented. The core application reads its own saved records. Integration configuration, authorization and sanitized response testing are future work; no successful response is claimed.

5.8 User Management and Role-Based Access

The implemented demonstration uses the owner account. Reporter, coordinator and repair-team responsibilities are an access-control design rather than tested saved profiles. The current Users settings state that the account email must be confirmed before users can be added. No credentials are included in this report. A production rollout would require configured permissions and independent tests with each intended user [4].

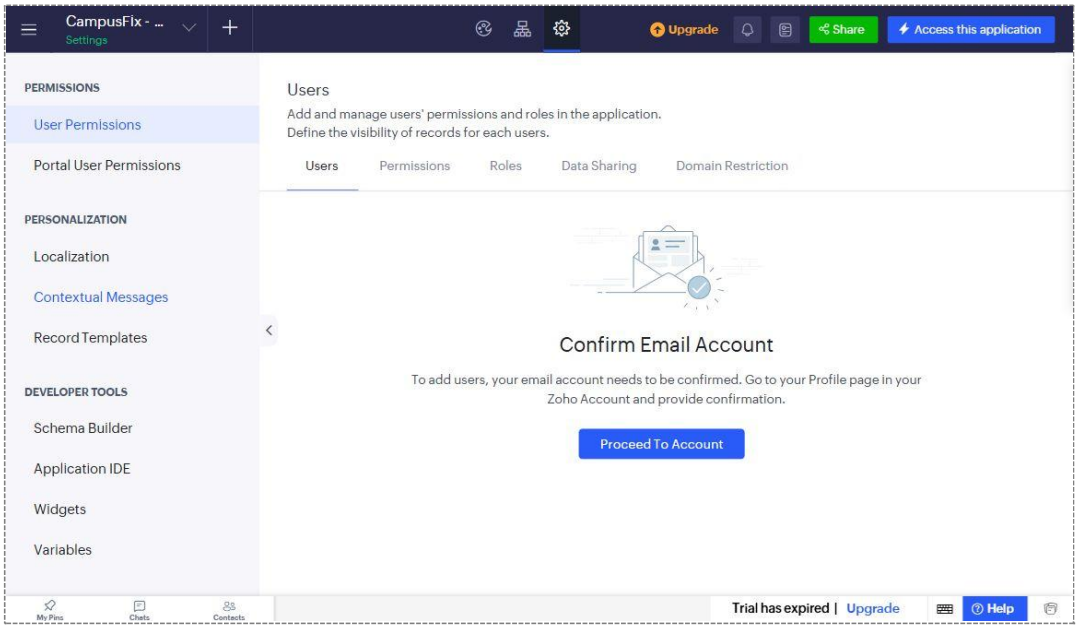


Figure 5.7: Current Users settings and email-confirmation prerequisite

5.9 AI and Analytics Features

The implemented analytics are the six count indicators and an independent reconciliation against the list and Kanban groups. Two of eight requests are resolved, which is 25 percent. No Zia prediction, AI field, pivot chart or Zoho Analytics connection was executed. AI assistance used during project preparation is acknowledged; it is not evidence of an in-application Zia feature.

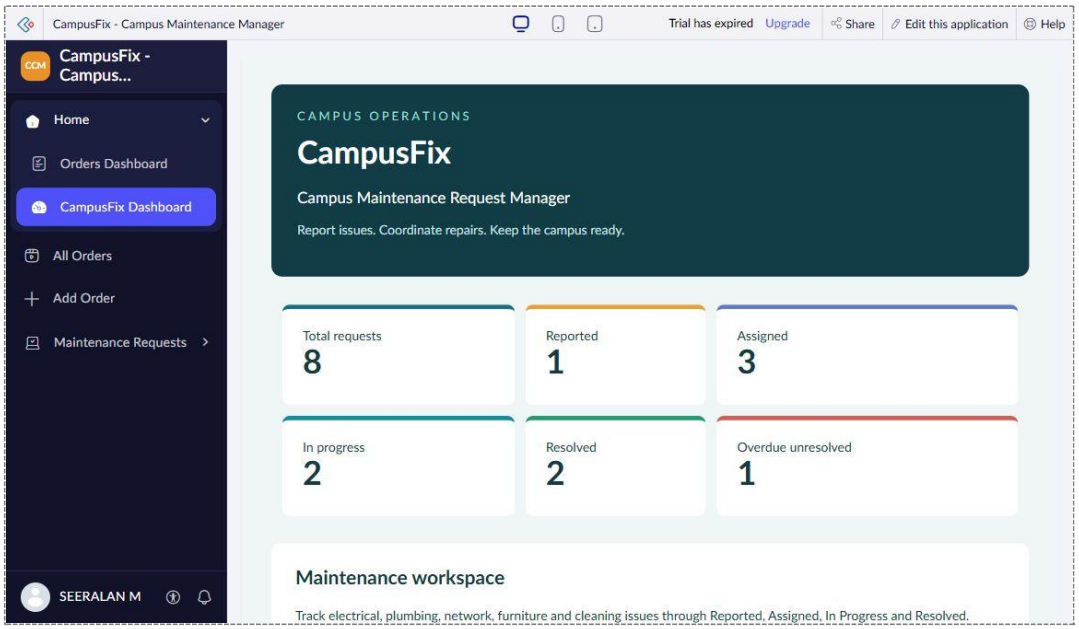


Figure 5.8: CampusFix count indicators used for reconciliation

5.10 Pro-Code Extension

No Catalyst service, JavaScript widget or reusable custom function was implemented. The validation script and HTML/Deluge page were sufficient for the current demonstration. A later extension could provide a tested notification or integration service, with authentication and error handling designed before release.

5.11 Mobile and Portal Access

> ***Mobile application and customer portal access were not implemented or tested. The demonstrated access channel is the authenticated desktop browser.***

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Functional testing used fictional request data. The initial build tested valid submission, duplicate references, five rejection conditions and an edited status reflected in dashboard counts. Subsequent inspection reconciled all eight records across the list, board and dashboard. Tests were performed through the owner browser session. Independent UAT, restricted-user enforcement, API behaviour, mobile layouts and performance under load remain unverified.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Valid submission	Complete fictional request	Record saved	Eight valid requests saved	Pass
TC-02	FR-02	Duplicate reference	Reuse CF-001	Reject duplicate	Unique-value error shown	Pass
TC-03	FR-03	Date order	Target earlier than reported	Reject	Date-order alert shown	Pass
TC-04	FR-03	Future report date	Use 01-Jan-2030	Reject	Future-date alert shown	Pass
TC-05	FR-04	Short description	Use Fan	Reject	Minimum-length alert shown	Pass
TC-06	FR-05	Missing team	Assigned with blank team	Reject	Team-required alert shown	Pass
TC-07	FR-05	Missing notes	Resolved with blank notes	Reject	Resolution-notes alert shown	Pass
TC-08	FR-06	List consistency	Open saved list	8 records	Showing 8 of 8	Pass
TC-09	FR-06	Board grouping	Inspect four status columns	1 / 3 / 2 / 2	Groups match saved records	Pass
TC-10	FR-07	Dashboard total	Reconcile status values	Total 8	1+3+2+2 = 8	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-11	FR-07	Saved status update	CF-008 Reported to Assigned	Counts update on reload	2/2 changed to 1/3	Pass
TC-12	FR-01	Default status	Open a new blank form	Reported	Reported shown	Pass
TC-13	FR-01	Category choices	Open Category choices	Five defined categories	Choices verified live	Pass
TC-14	FR-01	Priority choices	Open Priority choices	Low/Medium/High	Choices verified live	Pass
TC-15	FR-06	Record detail	Open CF-008	Saved team/status/dates	Assigned; Facilities Team shown	Pass
TC-16	NFR-02	Restricted-user test	Separate account/profile	Enforce allowed actions	Not executed	Pending
TC-17	NFR-01	Load benchmark	Representative larger dataset	Measured response times	Not executed	Pending
TC-18	NFR-03	Independent UAT	Two end-user sessions	Record feedback	Not executed	Pending

6.3 Debugging Deluge Scripts

The Deluge editor identified a field-link-name error for Assigned_Team during the build. The field label had changed locally while the saved link name had not persisted. The form properties were saved, the actual link names were verified in the live form, and the validation action compiled successfully. Submission alerts then provided direct feedback for the negative tests. Figure 6.1 shows a real rejected future-date entry.

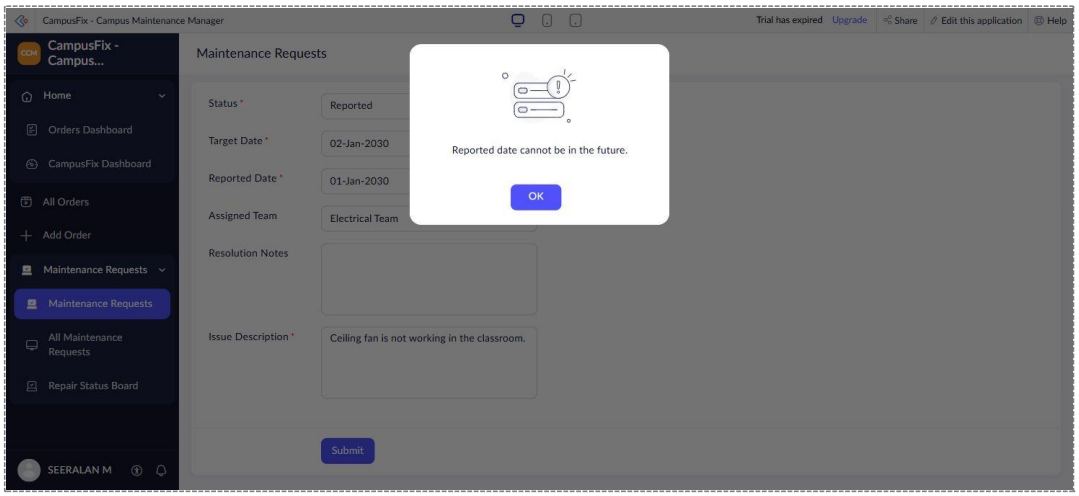


Figure 6.1: Actual rejection of a future reported date

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Assigned_Team link not recognized by script	Field rename had not persisted	Saved and rechecked field link names; action compiled	Closed
BUG-02	Dropdown choices not reflected in live form	Property edits had not persisted	Saved properties and verified live dropdown choices	Closed

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
End user 1	Not tested	No independent feedback collected	Arrange a supervised review
End user 2	Not tested	No independent feedback collected	Arrange a supervised review

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

CampusFix is accessible in the existing authenticated Creator application. The current header displays Trial has expired. No Development to Stage to Production release was executed, and no public portal was published. The repository contains component definitions and documentation; these are not a complete restorable application backup. A complete export, controlled restore test and release procedure are required before production deployment.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	No - owner-session functional tests only
2	Sample/test data removed or isolated	Fictional records identified as demo data
3	Roles, permissions and users configured in Production	No
4	Connections authorised in Production	Not applicable - none created
5	Schedules and workflows enabled	Validation workflow tested; no schedule
6	Application published and URL verified	Authenticated app URL verified; no staged release
7	Version / release notes recorded	Source and evidence documented; no release pipeline

7.2 Performance Optimisation

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Criteria-based count queries	Dashboard	No separate baseline measured	Six saved queries used	No timing improvement claimed
Status-based board grouping	Repair Status Board	No baseline measured	Four status groups shown	Functional organisation only
Future representative benchmark	Application	Not measured	Not measured	Pending

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Authenticated account access	Creator owner session	Limits unauthenticated access; independent security testing pending
Required and unique fields	Maintenance Requests	Incomplete and duplicate entries
Conditional validation	Submission workflow	Inconsistent dates and missing repair information
Personal-data designation	Contact Email field	Identifies contact data; does not prove role isolation or encryption

7.4 Governance and Administration

Components use consistent Maintenance_Requests and CampusFix names. The repository retains the saved form, report and workflow definitions, documentation and evidence. The exact dashboard implementation must also be retained for a complete rebuild; a wrapper referencing an HTML snippet alone is insufficient. Audit-trail review, scheduled backups and independent restore testing remain future administration tasks. The app settings currently show UTC and dd-MMM-yyyy; intended campus timezone boundaries should be confirmed before rollout.

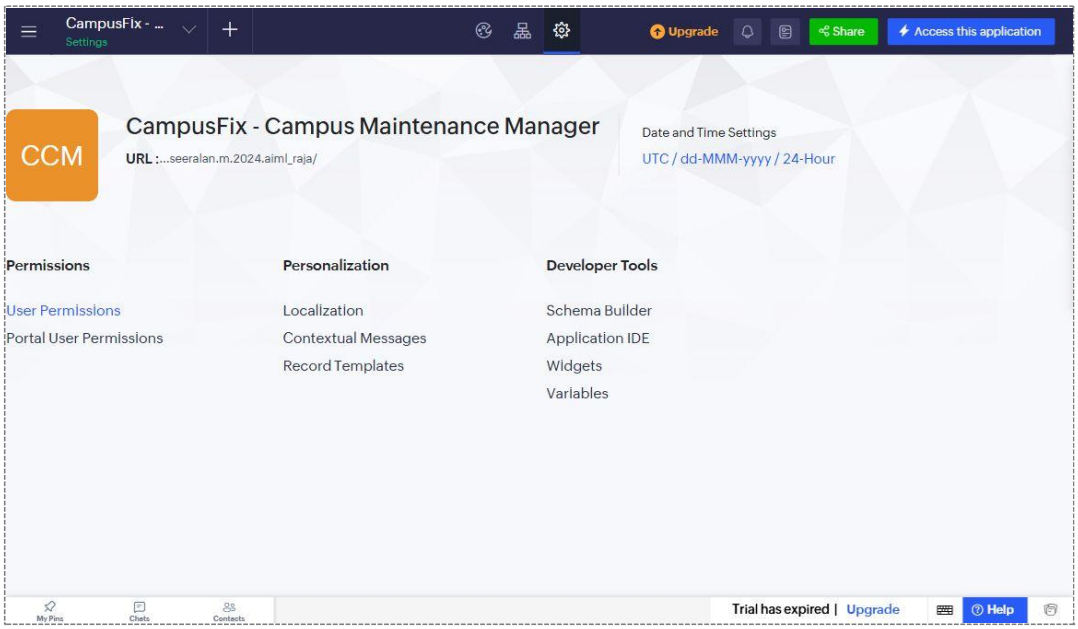


Figure 7.1: Current CampusFix application settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

CF-008 is the demonstrated desk-repair request. The form captures its location, category and description; the saved detail contains its target date and Facilities Team assignment. During the build, CF-008 was changed from Reported to Assigned and the dashboard counters changed from 2/2 to 1/3 for those statuses. The current board places it in Assigned. The walkthrough is one owner-session demonstration, not a multi-user approval process.

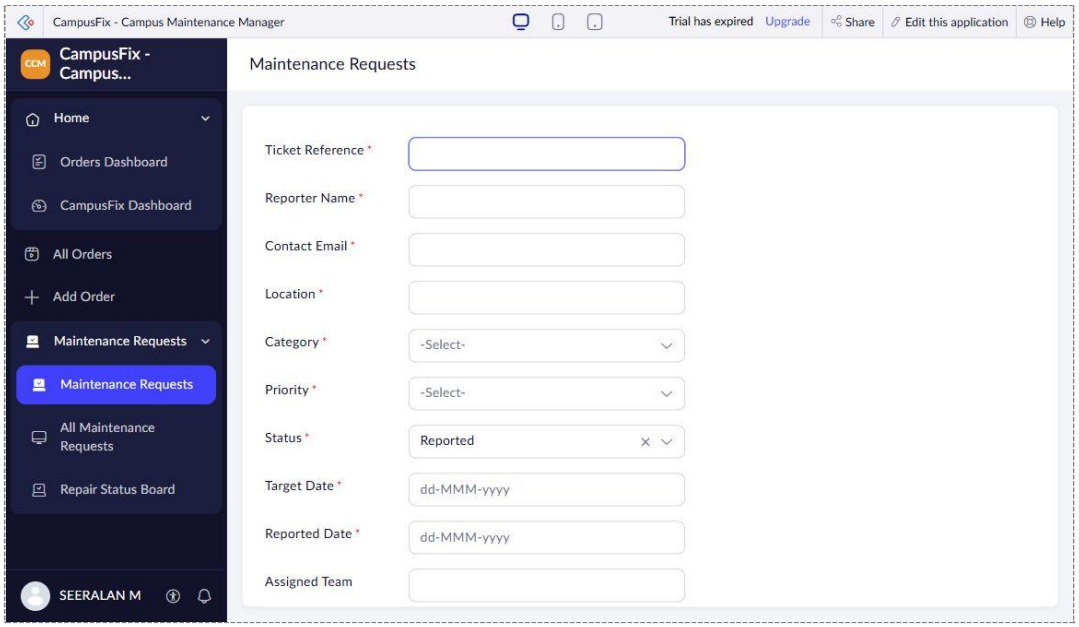


Figure 8.1: Maintenance Requests form for issue capture

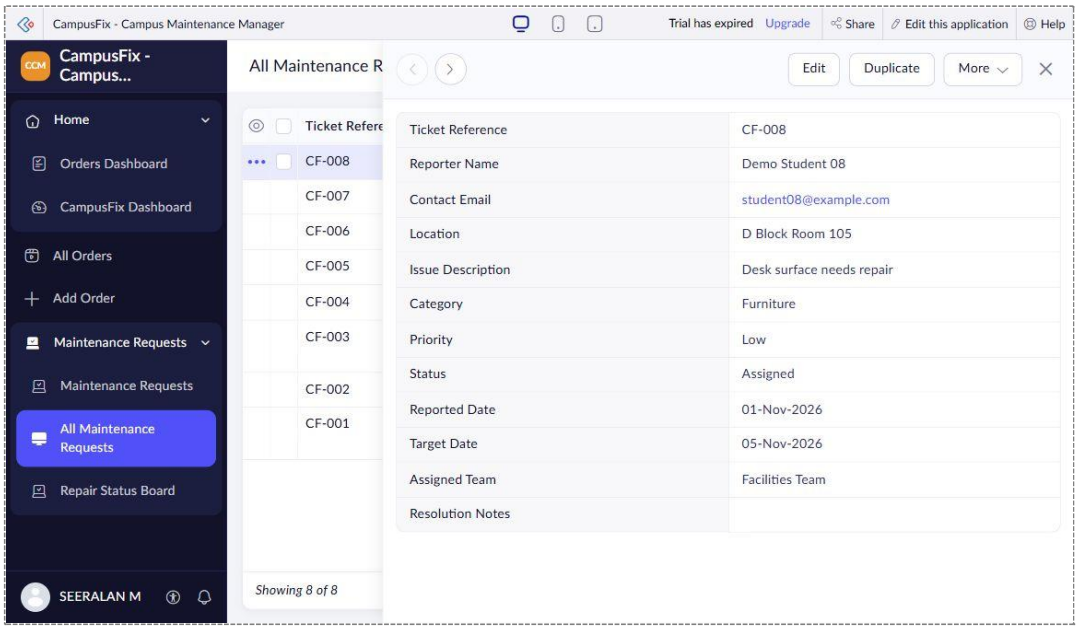


Figure 8.2: Saved CF-008 maintenance request details

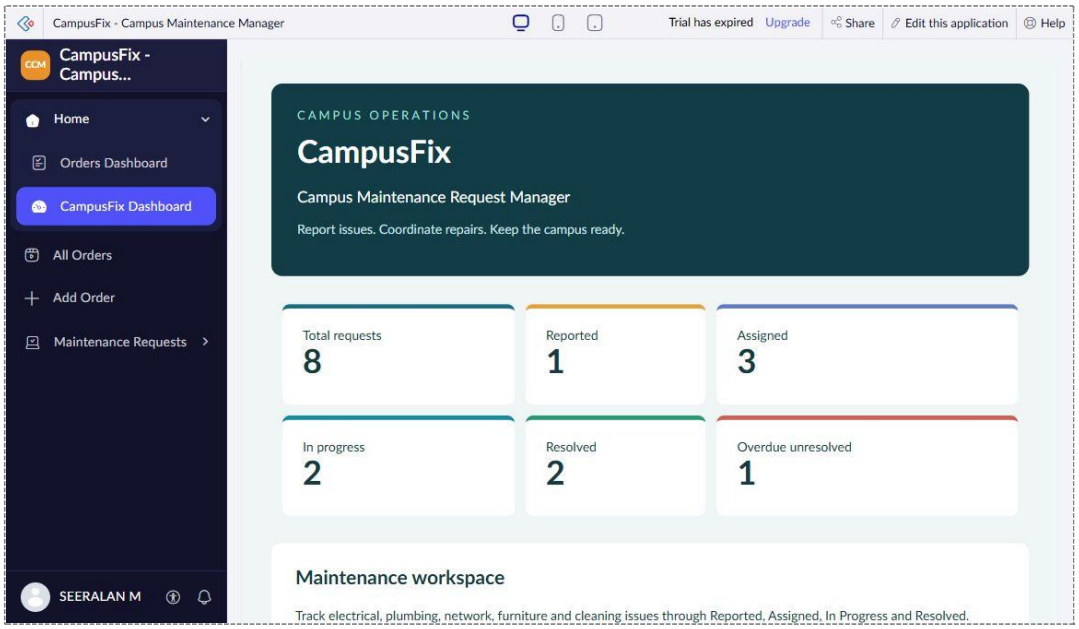


Figure 8.3: Dashboard after the saved status update

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Structured capture	5.2; Maintenance Requests	Twelve fields and eight saved requests
Five validation rules	5.5, 5.6; DS-01	Five negative tests rejected
Consistent views	5.3; list and Kanban	Eight records; groups 1/3/2/2

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Live dashboard counts	5.4; DS-02	Total 8 and update reflected after reload
Document tests and limits	6; 7; Appendix A	Passes and pending checks distinguished

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Request details	Illustrative separate messages	One saved form record	Structured information; no time saving measured
Status review	Illustrative follow-up messages	Kanban status groups	Current saved status visible
Summary preparation	Illustrative separate manual summary	Count queries on page load	No duplicate summary dataset
Access model	No measured baseline	Owner access verified only	Restricted roles still require testing

8.4 Limitations of the Low-Code Approach

The one-application subscription limit required the project to reuse an existing workspace. Original order components therefore remain visible in navigation but are excluded from maintenance counts. Field edits sometimes appeared locally before persisting, so link names and dropdown choices were checked again in live mode. The board may require horizontal scrolling. Separate roles, integration quotas, high-volume behaviour and complete export/restore have not been tested. These are concrete limits of the demonstrated scope rather than measured platform-wide limits.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Visual form and report configuration are the no-code portion of CampusFix. Deluge validation and aggregate queries form its low-code portion. The HTML dashboard adds manually authored presentation code. A future external service or custom widget would introduce a pro-code boundary and would need independent interface, security and deployment tests. The appropriate choice depends on the requirement, not on assuming that every feature must be scripted.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

CampusFix implements structured campus issue capture, a saved-record list, a status Kanban board and a workload dashboard in Zoho Creator. Its validation workflow rejects five invalid input conditions, and ticket references are unique. Eight fictional requests and the edited CF-008 record provide the working demonstration. The project establishes the core maintenance workflow while explicitly retaining unverified integrations, roles, performance and deployment activities as future work.

9.2 Future Enhancements

- Configure and test reporter, coordinator and repair-team permissions after account email confirmation.
- Add notification and external-summary integration only after authorization, failure handling and response tests are defined.
- Add a complete backup/restore procedure, timezone review, mobile checks, staged deployment and representative performance measurements.

9.3 Learning Outcomes and Career Relevance

The project develops skills in form modelling, report configuration, Deluge conditions, aggregate queries, debugging, screenshot evidence and technical documentation. These skills support entry-level low-code development, solutions analysis and automation work. Further learning should cover access control, reliable integrations, release management and independent acceptance testing. Understanding the limits of the current demonstration is part of presenting it accurately.

REFERENCES

[1] Zoho Corporation, “Understand Applications,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/creating-applications/understand-applications/articles/understand-application (accessed Nov. 2, 2026).

[2] Zoho Corporation, “Understanding form builder,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/forms/the-concept-of-forms/articles/form-builder (accessed Nov. 2, 2026).

[3] Zoho Corporation, “Understanding workflows,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/understand-workflows/articles/understand-workflows (accessed Nov. 2, 2026).

[4] Zoho Corporation, “Understanding permissions,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/application-settings/manage-and-configure-user-permissions/articles/understand-permissions (accessed Nov. 2, 2026).

[5] Zoho Corporation, “Count,” Zoho Deluge Help. [Online]. Available: https://www.zoho.com/deluge/help/data-access/aggregate-records/count.html (accessed Nov. 2, 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Existing app and CampusFix menu	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	N	Not implemented / not verified	5.2	Not applicable
II	Lookup field (relationship between forms)	N	Not implemented / not verified	4.2, 5.2	Not applicable
II	Subform (parent–child data)	N	Not implemented / not verified	4.2, 5.2	Not applicable
II	Field validations / layout rules	Y	Form properties and DS-01	5.2	Code 5.1
II	List report with filter, grouping, custom action	N	Not implemented / not verified	5.3	Not applicable
II	Kanban / Calendar / Timeline / Map report	Y	Repair Status Board	5.3	Fig. 5.4
II	Pivot table / chart report	N	Not implemented / not verified	5.3	Not applicable
II	Page / dashboard built with UI Builder	Y	CampusFix Dashboard	5.4	Fig. 5.5
II	Deluge – form events (validate / success / field actions)	Y	DS-01 On Validate	5.5	Code 5.1
II	Deluge – functions, fetch, loops, collections	N	Not implemented / not verified	5.5	Not applicable
II	Email / in-app notifications	N	Not implemented / not verified	5.5, 5.6	Not applicable
II	Form workflow	Y	Validate Maintenance Requests	5.6	Fig. 5.6
II	Scheduled workflow	N	Not implemented / not verified	5.6	Not applicable
II	Approval process	N	Not implemented / not verified	5.6	Not applicable
II	Blueprint	N	Not implemented / not verified	5.6	Not applicable

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Batch / report workflow	N	Not implemented / not verified	5.6	Not applicable
II	Testing & debugging evidence	Y	Negative tests and defect log	6	Fig. 6.1; Table 6.1
III	Integration with a Zoho service	N	Not implemented / not verified	5.7	Not applicable
III	Connection / REST API call (invokeurl)	N	Not implemented / not verified	5.7	Not applicable
III	Webhook or Creator REST API	N	Not implemented / not verified	5.7	Not applicable
III	Users, roles and permission profiles	N	Not implemented / not verified	5.8	Not applicable
III	Record-level access criteria / customer portal	N	Not implemented / not verified	5.8	Not applicable
III	Zia / AI feature	N	Not implemented / not verified	5.9	Not applicable
III	Analytics (charts / Zoho Analytics)	N	Not implemented / not verified	5.9	Not applicable
IV	Performance optimisation applied & measured	N	Not implemented / not verified	7.2	Not applicable
IV	Deployment across environments	N	Not implemented / not verified	7.1	Not applicable
IV	Security controls (encryption, OAuth, permissions)	N	Not implemented / not verified	7.3	Not applicable
IV	Governance (audit trail, administration)	N	Not implemented / not verified	7.4	Not applicable
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not implemented / not verified	5.10	Not applicable
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum reflection	8.4, 8.5	Sections 8.4-8.5
—	Mobile app / portal access	N	Not implemented / not verified	5.11	Not applicable

Total features fully evidenced against the combined checklist criteria: 8 / 32. A negative entry may include a partially implemented component; the corresponding section explains the scope.

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	2	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	2	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	1	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	1	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Maintenance Requests validation workflow

```
// DS-01 | Maintenance Requests | Created or Edited
// Event: Validations on form submission
if(input.Target_Date < input.Reported_Date)
{
    alert "Target date must be on or after the reported date.";
    cancel submit;
}
if(input.Reported_Date > zoho.currentdate)
{
    alert "Reported date cannot be in the future.";
    cancel submit;
}
if(ifnull(input.Issue_Description,"").trim().length() < 10)
{
    alert "Please describe the issue using at least 10 characters.";
    cancel submit;
}
if(input.Status != "Reported" &&
   ifnull(input.Assigned_Team,"").trim() == "")
{
    alert "Assign a repair team before progressing this request.";
    cancel submit;
}
if(input.Status == "Resolved" &&
   ifnull(input.Resolution_Notes,"").trim() == "")
{
    alert "Resolution notes are required before closing a request.";
    cancel submit;
}
```

C.2 CampusFix Dashboard aggregate queries

```
// DS-02 | CampusFix Dashboard HTML snippet
// Event: page load | no record changes
total = Maintenance_Requests[ID != 0].count();
reported = Maintenance_Requests[Status == "Reported"].count();
assigned = Maintenance_Requests[Status == "Assigned"].count();
progress = Maintenance_Requests[Status == "In Progress"].count();
resolved = Maintenance_Requests[Status == "Resolved"].count();
overdue = Maintenance_Requests[
    Target_Date < zoho.currentdate && Status != "Resolved"
].count();
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application URL in an internet-connected browser and sign in with the authorized Zoho account. Select CampusFix Dashboard under Home. Access remains governed by Creator permissions; no public or embedded credentials are supplied.

D.2 Instructions for the Owner Demonstration

- Select Report an issue. Enter a new unique reference, fictional reporter and email, location, category, priority, reported and target dates, and a description of at least ten characters. Submit valid data.
- Open All Maintenance Requests, inspect the new record, choose Edit, assign a team and update its status. Enter resolution notes before selecting Resolved. Use Update to save the change.

D.3 Instructions for Reviewing Reports

- Open Repair Status Board to see the four configured status columns. Scroll horizontally if needed. Open a card to inspect its details.
- Return to CampusFix Dashboard to recalculate the six indicators. Reconcile the status totals with the list count and explain that overdue is a subset, not an additional status.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Seeralan M	Register No.	2116241501193
Project Title	CampusFix Campus Maintenance Manager	Date	_____

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Seeralan M	Register No.	2116241501193
Department / Section	AIML	Date of Viva	_____
Application URL Verified	For examiner verification	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____